

리눅스 1 - CFS & EEVDF

남궁명수

[CFS와 EEVDF 스케줄링]

[1] [CFS(Completely Fair Scheduler)]

CFS는 Completely Fair Scheduler, 즉 완전 공정 스케줄러라는 뜻이다.
이름처럼 CPU 자원을 모든 프로세스에 최대한 공평하게 분배하는 것을 목표로 한다.

정확히는 모든 프로세스에 동일한 CPU 시간을 주는 것이 아니라,
프로세스의 우선순위인 nice 값에 비례하여 공정한 CPU 시간을 제공한다.

CFS는 우선순위에 따른 가중치와 가상 실행 시간인 vruntime을 기반으로,
실행 대기 중인 프로세스 가운데 vruntime이 가장 작은 프로세스에 CPU를 할당한다.

리눅스 커널의 실제 스케줄링 대상은 프로세스보다 더 정확하게는 **실행 가능한 태스크, 즉 스레드**이다.
다만 설명에서는 편의상 프로세스라고 표현한다.

[1.1] CFS의 핵심 개념

[가상 실행 시간(vruntime)]

vruntime은 프로세스가 실제로 사용한 CPU 시간에 우선순위 가중치를 반영한 가상의 실행 시간이다.

vruntime이 작다

- 상대적으로 CPU를 적게 사용했다.
- 다음에 실행될 가능성이 높다.

vruntime이 크다

- 상대적으로 CPU를 많이 사용했다.
- 다른 프로세스에 CPU를 양보한다.

우선순위가 동일한 프로세스라면 실제 실행 시간과 vruntime이 비슷한 속도로 증가한다.

프로세스 A가 5ms 실행

- A의 vruntime도 약 5ms 증가

하지만 nice 값이 다르면 vruntime이 증가하는 속도도 달라진다.

[nice 값과 가중치]

리눅스 일반 프로세스의 우선순위는 nice 값으로 표현한다.

nice 값의 범위: -20 ~ 19

nice 값	우선순위	가중치	vruntime 증가 속도	CPU 할당량
-20	매우 높음	큼	느림	많음
0	기본	기본	보통	보통
19	매우 낮음	작음	빠름	적음

nice 값이 낮은 프로세스는 가중치가 크기 때문에 vruntime이 천천히 증가한다.
따라서 다시 가장 작은 vruntime을 가지기 쉬워 CPU를 더 많이 할당받는다.

반대로 nice 값이 높은 프로세스는 가중치가 작아 vruntime이 빠르게 증가하므로 CPU를 적게 할당받는다.

nice 값이 낮음
→ 우선순위가 높음
→ 가중치가 큼
→ vruntime이 천천히 증가
→ CPU를 더 많이 할당받음

[1.2] CFS의 동작 방식

[1단계: 프로세스가 실행 가능 상태가 된다]

프로세스가 생성되거나 I/O 작업을 마치고 깨어나면 RUNNABLE 상태가 되어 CPU의 실행 큐에 들어간다.

프로세스 생성 또는 I/O 완료
↓
RUNNABLE 상태
↓
CPU 실행 큐에 등록

[2단계: nice 값에 따른 가중치를 확인한다]

CFS는 프로세스의 nice 값을 가중치로 변환한다.

nice 값이 낮음 → 가중치가 큼
nice 값이 높음 → 가중치가 작음

이 가중치는 프로세스가 장기적으로 어느 정도의 CPU 시간을 받을지 결정한다.

[3단계: 실행 대기 프로세스를 Red-Black Tree에 관리한다]

전통적인 CFS는 실행 가능한 프로세스를 vruntime 순서로 Red-Black Tree에 저장한다.



트리의 왼쪽에는 vruntime이 작은 프로세스가, 오른쪽에는 vruntime이 큰 프로세스가 위치한다.

Red-Black Tree를 사용하면 프로세스 삽입과 삭제, 다음 실행 후보 관리 등을 일반적으로 $O(\log N)$ 시간에 처리할 수 있다.

[4단계: vruntime이 가장 작은 프로세스를 선택한다]

CFS는 트리에서 vruntime이 가장 작은 프로세스를 찾아 CPU를 할당한다.

예를 들어 다음과 같은 프로세스가 있다고 하자.

프로세스	vruntime
A	20
B	10
C	15

B는 지금까지 CPU를 상대적으로 가장 적게 사용했으므로 다음 실행 프로세스로 선택된다.

다음 실행 프로세스: B

[5단계: 프로세스를 일정 시간 실행한다]

선택된 프로세스는 계산된 실행 시간 동안 CPU를 사용한다.

CFS는 Round Robin처럼 모든 프로세스에 완전히 동일한 고정 시간만 제공하지 않는다. 실행 가능한 프로세스의 수와 각 프로세스의 가중치를 반영해 CPU 실행 시간을 분배한다.

개념적으로 다음과 같다.

프로세스 실행 시간
 $\approx \text{전체 기준 시간} \times \text{프로세스 가중치} / \text{전체 프로세스 가중치}$

우선순위가 같은 프로세스가 3개라면 CPU 시간을 비슷하게 나눠 가진다.

A: 약 1/3
B: 약 1/3
C: 약 1/3

A의 우선순위가 더 높다면 A가 더 많은 CPU 시간을 받는다.

A: 50%

B: 25%

C: 25%

[6단계: 실행한 시간만큼 vruntime을 갱신한다]

프로세스가 CPU를 사용하면 실제 실행 시간과 가중치를 이용해 vruntime을 증가시킨다.

개념적으로 다음과 같이 계산한다.

vruntime 증가량

= 실제 실행 시간 × 기본 가중치 / 프로세스 가중치

프로세스 B가 실행되어 vruntime이 10에서 18로 증가했다고 하자.

[실행 전]

B(10) → C(15) → A(20)

[B 실행 후]

C(15) → B(18) → A(20)

이제 C의 vruntime이 가장 작기 때문에 C가 다음 실행 후보가 된다.

[7단계: 필요하면 현재 프로세스를 선점한다]

현재 프로세스가 충분한 CPU 시간을 사용했고 다른 프로세스가 더 작은 vruntime을 가지고 있다면, 스케줄러는 현재 프로세스의 실행을 중단하고 다른 프로세스에 CPU를 할당한다.

B 실행

↓

B의 vruntime 증가

↓

C의 vruntime이 더 작아짐

↓

B를 중단하고 C 실행

이 과정을 반복하면서 각 프로세스가 우선순위에 비례하는 공정한 CPU 시간을 받게 한다.

[1.3] CFS 동작 예시

nice 값이 동일한 A, B, C가 있다고 하자.

초기 상태

프로세스	vruntime
A	0
B	0
C	0

A가 3ms 실행된다.

프로세스	vruntime
A	3
B	0
C	0

B와 C가 CPU를 적게 받았으므로 다음에는 B가 실행된다.

프로세스	vruntime
A	3
B	3
C	0

이제 C가 실행된다.

프로세스	vruntime
A	3
B	3
C	3

결국 세 프로세스가 동일하게 CPU를 사용한다.

A 실행 → B 실행 → C 실행

이후에도 같은 과정이 반복된다.

A → B → C → A → B → C

실제 실행 순서가 항상 이렇게 고정되는 것은 아니지만, 장기적으로는 각 프로세스가 우선순위에 맞는 CPU 시간을 받게 된다.

[1.4] CFS의 장점

공정한 CPU 분배

프로세스가 지금까지 받은 CPU 시간을 `vruntime`으로 관리하기 때문에 특정 일반 프로세스가 CPU를 계속 독점하는 것을 방지한다.

우선순위가 같다면 장기적으로 비슷한 CPU 시간을 받고, 우선순위가 다르면 가중치에 비례해 CPU 시간을 받는다.

우선순위를 비율로 반영

CFS의 `nice` 값은 단순히 누가 먼저 실행되는지만 결정하는 것이 아니다.

가중치를 통해 프로세스가 장기적으로 받을 CPU 시간의 비율에 영향을 준다.

높은 우선순위

→ `vruntime`이 천천히 증가

→ 더 자주 선택

→ 더 많은 CPU 시간 확보

기아 상태 완화

특정 프로세스가 오랫동안 실행되지 않으면 다른 프로세스가 실행되는 동안 상대적으로 `vruntime`이 작게 유지된다.

따라서 언젠가는 실행 후보가 되어 CPU를 받을 가능성이 커지므로 일반적인 우선순위 스케줄링보다 기아 상태가 발생할 가능성이 작다.

단, CPU 제한이나 실시간 작업, `cgroup` 설정 등 다른 조건이 개입하면 일반 작업이 크게 지연될 가능성은 있다.

프로세스 수 증가에 대응 가능

Red-Black Tree를 이용해 실행 가능한 프로세스를 관리하므로 프로세스의 삽입과 삭제를 효율적으로 처리할 수 있다. 프로세스가 많아져도 단순한 전체 순회보다 확장성이 좋다.

대화형 작업과 CPU 집중 작업을 함께 처리

CFS는 CPU를 계속 사용하는 계산 작업과 I/O를 기다렸다가 짧게 실행되는 대화형 작업을 함께 관리할 수 있다. 잠자다가 깨어난 대화형 작업도 실행 큐의 현재 기준에 맞게 배치되어 비교적 빠르게 CPU를 받을 수 있다.

[1.5] CFS의 단점

응답시간을 직접 보장하지 않는다

CFS의 가장 중요한 목표는 공정성이다.

따라서 특정 프로세스가 반드시 몇 ms 안에 실행되어야 한다는 실시간 응답시간을 보장하지 않는다.

공정하게 CPU를 분배하는 것 $\neq$ 정해진 시간 안에 반드시 실행하는 것

엄격한 실행 기한이 필요한 작업은 `SCHED_FIFO`, `SCHED_RR`, `SCHED_DEADLINE` 등의 별도 스케줄링 정책을 사용해야 한다.

짧고 급한 작업을 충분히 구분하기 어렵다

전통적인 CFS는 주로 `vruntime`이 가장 작은 프로세스를 선택한다.

따라서 키보드 입력 처리처럼 짧고 빠르게 끝나야 하는 작업과, 파일 압축처럼 오래 실행되는 작업의 긴급성을 세밀하게 구분하는 데 한계가 있다.

이러한 지연시간 문제를 개선하기 위해 최신 리눅스에서는 `EEVDF` 방식이 도입됐다.

프로세스가 많으면 개별 실행 시간이 짧아질 수 있다

실행 가능한 프로세스가 많아지면 CPU 시간을 여러 프로세스가 나눠야 한다.

프로세스 2개 → 각 프로세스가 비교적 오래 실행
프로세스 100개 → 각 프로세스가 받을 실행 시간이 짧아짐

실행 전환이 너무 자주 발생하면 문맥 교환 비용과 캐시 미스가 증가할 수 있다.

멀티코어에서는 부하 분산 비용이 발생한다

멀티코어 환경에서는 CPU별로 실행 큐가 존재한다.

특정 CPU에 작업이 몰리면 다른 CPU로 작업을 이동하는 Load Balancing이 필요하다.

CPU 0: A, B, C
CPU 1: 없음
↓ 작업 이동
CPU 0: A, B
CPU 1: C

작업을 이동하면 기존 CPU 캐시를 제대로 활용하지 못해 캐시 미스와 작업 이주 비용이 발생할 수 있다.

설정이 복잡하다

nice 값뿐만 아니라 다음 요소가 실제 CPU 분배에 영향을 줄 수 있다.

- cgroup CPU 가중치
- CPU affinity
- 멀티코어 Load Balancing
- NUMA 구조
- 스케줄링 정책
- 실행 가능한 프로세스 수

따라서 실제 서버에서 발생하는 스케줄링 결과를 vruntime 하나만으로 설명하기 어려울 수 있다.

[2] [EEVDF(Earliest Eligible Virtual Deadline First)]

EEVDF는 Earliest Eligible Virtual Deadline First의 약자다.

다음과 같이 해석할 수 있다.

CPU를 받을 자격이 있는 프로세스 중 가상 마감 시간이 가장 빠른 프로세스를 먼저 실행한다.

리눅스는 Linux 6.6부터 일반 프로세스의 작업 선택 방식을 기존 CFS 방식에서 EEVDF 방식으로 전환하기 시작했다. EEVDF는 CFS의 공정성 원리를 완전히 버린 새로운 스케줄러가 아니다.

CFS가 사용하던 nice 값, 가중치, vruntime 기반의 공정성을 유지하면서, 다음 프로세스를 선택하는 방식을 더욱 정교하게 개선한 것이다.

[2.1] EEVDF가 필요한 이유

전통적인 CFS는 `vruntime`이 가장 작은 프로세스를 선택한다.

이 방식은 CPU 시간을 공정하게 나누는 데는 효과적이지만, 어떤 프로세스가 더 빠르게 응답해야 하는지를 충분히 표현하기 어렵다.

예를 들어 다음 두 프로세스가 있다고 하자.

프로세스 A: 대용량 파일 압축

프로세스 B: 사용자의 키보드 입력 처리

A는 CPU를 오래 사용해야 하는 계산 작업이다.

B는 실행 시간은 짧지만 빨리 실행되어야 사용자 화면이 즉시 반응한다.

CFS는 주로 다음을 판단한다.

누가 지금까지 CPU를 적게 사용했는가?

EEVDF는 다음 두 가지를 판단한다.

1. 누가 자신의 공정한 CPU 시간을 덜 받았는가?
2. 그중 누가 더 빨리 실행되어야 하는가?

이를 위해 `lag`와 `Virtual Deadline`을 사용한다.

[2.2] EEVDF의 핵심 개념

[Lag]

`lag`는 프로세스가 공정하게 받아야 할 CPU 시간과 실제로 받은 CPU 시간의 차이를 나타낸다.

`lag`가 양수

→ 받아야 할 CPU 시간보다 적게 받음

→ CPU를 더 받아야 함

`lag`가 음수

→ 받아야 할 CPU 시간보다 많이 받음

→ 다른 프로세스에 CPU를 양보해야 함

예를 들어 모든 프로세스가 공정하게 10ms씩 받아야 한다고 하자.

프로세스	받아야 할 시간	실제 받은 시간	lag	의미
A	10ms	6ms	+4ms	CPU를 덜 받음
B	10ms	12ms	-2ms	CPU를 많이 받음
C	10ms	9ms	+1ms	CPU를 조금 덜 받음

A와 C는 공정한 몫보다 CPU를 적게 받았으므로 CPU를 받을 자격이 있다.

B는 이미 자신의 몫보다 CPU를 많이 받았으므로 현재는 CPU를 받을 자격이 없다.

[Eligible]

Eligible은 CPU를 할당받을 **자격이 있는 상태**를 뜻한다.

단순화하면 lag가 0 이상인 프로세스를 실행 후보로 판단한다.

A lag: +4 → 실행 자격 있음
B lag: -2 → 실행 자격 없음
C lag: +1 → 실행 자격 있음

이 단계에서는 A와 C가 실행 후보가 된다.

전체 프로세스: A, B, C
실행 후보: A, C

[Virtual Deadline]

Virtual Deadline은 프로세스가 가상 시간 기준으로 언제까지 CPU 실행 기회를 받아야 하는지를 나타내는 값이다.

이 값은 현실에서 사용하는 실제 마감 시간이 아니다.

오후 3시까지 작업 완료

같은 실제 시간이 아니라, 스케줄러 내부에서 공정성과 응답시간을 판단하기 위한 가상의 마감 시간이다. 프로세스의 가상 시작 시간과 할당된 실행 시간인 slice 등을 바탕으로 계산된다.

개념적으로는 다음과 같다.

Virtual Deadline
= Virtual Start Time + Virtual Slice

EEVDF는 실행 자격이 있는 프로세스 중 Virtual Deadline이 가장 빠른 프로세스를 선택한다.

[2.3] EEVDF의 동작 방식

[1단계: 실행 가능한 프로세스를 실행 큐에 등록한다]

프로세스가 생성되거나 I/O 작업을 완료하면 실행 가능 상태가 되어 실행 큐에 들어간다.

프로세스 생성 또는 I/O 완료
↓
RUNNABLE 상태
↓
Fair Scheduler 실행 큐 등록

[2단계: nice 값과 가중치를 반영한다]

EEVDF도 기존 CFS처럼 nice 값에 따른 가중치를 사용한다.

nice 값 낮음

→ 가중치 큼

→ 더 많은 CPU 시간 확보

nice 값 높음

→ 가중치 작음

→ 더 적은 CPU 시간 확보

따라서 EEVDF에서도 우선순위에 비례한 CPU 분배가 유지된다.

[3단계: vruntime과 lag를 계산한다]

프로세스가 지금까지 CPU를 얼마나 사용했는지 vruntime으로 관리한다.

이를 바탕으로 각 프로세스가 자신의 공정한 CPU 몫보다 덜 받았는지 또는 더 받았는지를 나타내는 lag를 계산한다.

공정한 몫보다 덜 실행

→ 양수 lag

공정한 몫보다 더 실행

→ 음수 lag

[4단계: 실행 자격을 판단한다]

EEVDF는 lag를 이용해 CPU를 받을 자격이 있는 프로세스를 먼저 찾는다.

프로세스	lag	실행 자격
A	+4	있음
B	-2	없음
C	+1	있음

이 경우 A와 C만 다음 실행 후보가 된다.

[5단계: Virtual Deadline을 비교한다]

실행 자격이 있는 프로세스들의 Virtual Deadline을 비교한다.

프로세스	lag	Virtual Deadline	실행 후보
A	+4	50	가능
B	-2	30	불가능
C	+1	40	가능

B의 Virtual Deadline이 가장 빠르지만 lag가 음수이므로 현재 실행 후보가 아니다.

실행 자격이 있는 A와 C 중에서는 C의 Virtual Deadline이 더 빠르다.

A의 Deadline: 50

C의 Deadline: 40

다음 실행 프로세스: C

[6단계: 선택한 프로세스를 실행한다]

선택된 프로세스는 자신에게 배정된 slice 동안 실행된다.

프로세스가 실행되면 다음 값들이 갱신된다.

실제 CPU 실행 시간

vruntime

lag

Virtual Deadline

프로세스가 CPU를 사용하면서 자신의 공정한 몫을 채우면 lag가 감소한다.

실행 전 lag: +4

CPU 사용

실행 후 lag: 0 또는 음수

그러면 다른 프로세스가 CPU를 받을 차례가 된다.

[7단계: 더 빠른 Deadline을 가진 프로세스가 들어오면 선점할 수 있다]

현재 A가 실행 중인데 새로 실행 가능해진 B의 Virtual Deadline이 더 빠르다면 B가 A를 선점할 수 있다.

A 실행 중

↓

B가 I/O 작업을 마치고 깨어남

↓

B가 실행 자격을 가짐

↓

B의 Virtual Deadline이 A보다 빠름

↓

A를 선점하고 B 실행

이러한 방식으로 짧고 빠른 응답이 필요한 작업의 지연시간을 줄일 수 있다.

[2.4] EEVDF 동작 예시

다음 세 프로세스가 있다고 하자.

프로세스	작업	lag	Virtual Deadline
A	파일 압축	+3	60
B	키보드 입력 처리	+1	30
C	기존 계산 작업	-2	20

[실행 자격 판단]

A: lag +3 → 자격 있음

B: lag +1 → 자격 있음

C: lag -2 → 자격 없음

C의 Deadline이 20으로 가장 빠르지만 이미 CPU를 많이 사용했으므로 선택할 수 없다.

[Deadline 비교]

A Deadline: 60

B Deadline: 30

B의 Deadline이 더 빠르므로 B가 먼저 실행된다.

키보드 입력 처리 B 실행

↓

짧은 시간 안에 입력 처리 완료

↓

그다음 파일 압축 A 실행

이렇게 EEVDF는 CPU 시간을 공정하게 분배하면서도 짧고 응답이 중요한 작업을 더 빠르게 처리할 수 있다.

[2.5] 잠자는 프로세스의 lag 처리

프로세스가 I/O를 기다리거나 sleep()을 호출하면 일정 시간 동안 CPU를 사용하지 않는다.

만약 잠자는 동안 받지 못한 CPU 시간을 계속 모두 축적하면,
프로세스가 깨어났을 때 지나치게 큰 양수 lag를 가질 수 있다.

오랫동안 Sleep

↓

받지 못한 CPU 시간 계속 축적

↓

깨어난 뒤 CPU를 지나치게 많이 사용

이를 악용하면 프로세스가 잠깐씩 잠들면서 CPU를 더 많이 받을 수도 있다.

EEVDF는 잠든 프로세스를 deferred dequeue 방식으로 처리하며, 해당 프로세스의 lag가 가상 시간에 따라 점진적으로 감소하도록 관리한다.

따라서 다음 두 가지를 함께 고려한다.

- 잠깐 잠들었다가 깨어난 대화형 작업에는 빠른 응답 제공
- 오랫동안 잠든 작업이 CPU 사용 권리를 과도하게 축적하는 것은 방지

[deferred queue]

EEVDF의 deferred dequeue는 CPU를 과도하게 사용한 태스크가 잠깐 잠들었다 깨어나는 방식으로 음수 lag를 없애지 못하도록, 해당 태스크를 런큐에 회계상 남겨 lag가 0으로 감쇠할 때까지 제거를 미루는 방식이다.

[2.6] EEVDF의 장점

CFS의 공정성을 유지한다

EEVDF도 nice 값, 가중치, vruntime을 사용한다.

따라서 우선순위에 비례해 CPU 시간을 분배하는 CFS의 공정성 원리를 유지한다.

대화형 작업의 응답성을 개선한다

키보드 입력, 마우스 입력, GUI 이벤트처럼 짧고 빠르게 실행되어야 하는 작업은 더 가까운 Virtual Deadline을 가질 수 있다. 따라서 CPU 집중 작업과 함께 실행되더라도 비교적 빠르게 CPU를 받을 수 있다.

짧은 실행 시간이 필요한 작업에 유리하다

짧은 slice를 가진 작업은 이른 Virtual Deadline을 가질 수 있어 빠르게 실행될 가능성이 높다.

짧은 작업을 먼저 실행

→ 빠르게 완료

→ 응답시간 감소

공정성과 지연시간을 함께 고려한다

CFS는 주로 vruntime을 통해 공정성을 판단했다.

EEVDF는 lag로 공정성을 확인하고 Virtual Deadline으로 실행 순서를 결정한다.

lag

→ CPU를 받을 자격과 공정성 판단

Virtual Deadline

→ 실행 자격이 있는 작업의 순서 결정

따라서 공정성과 응답성을 동시에 개선할 수 있다.

수면을 이용한 CPU 독점 방지

잠든 프로세스의 lag를 그대로 무한히 보존하지 않고 점진적으로 감소시킨다.

프로세스가 의도적으로 짧은 수면을 반복해 CPU 실행 권리를 과도하게 확보하는 것을 방지한다.

[2.7] EEVDF의 단점

기존 CFS보다 동작 원리가 복잡하다

기존 CFS는 다음 한 문장으로 핵심을 설명할 수 있었다.

vruntime이 가장 작은 프로세스를 실행한다.

EEVDF는 다음 요소를 모두 고려한다.

- vruntime
- lag
- 실행 자격
- Virtual Deadline
- slice
- 잠든 프로세스의 lag 감소

따라서 구현과 분석이 더 복잡하다.

실시간 실행을 보장하지 않는다

이름에 Deadline이 들어가지만 EEVDF의 Virtual Deadline은 실제 시간 제한을 의미하지 않는다.

따라서 다음과 같은 보장은 하지 않는다.

이 프로세스는 반드시 5ms 안에 실행된다.

엄격한 실시간 보장이 필요하다면 SCHED_DEADLINE이나 실시간 스케줄링 정책을 사용해야 한다.

튜닝에 따라 성능 결과가 달라질 수 있다

프로세스의 slice와 시스템 설정, 작업 특성에 따라 처리량과 응답시간의 결과가 달라질 수 있다.

짧은 slice를 지나치게 많이 사용하면 선점과 문맥 교환이 증가할 수 있고,
긴 slice를 사용하면 대화형 작업의 응답성이 나빠질 수 있다.

멀티코어 부하 분산 문제는 별도로 존재한다

EEVDF는 주로 각 CPU 실행 큐에서 다음 작업을 선택하는 문제를 개선한다.

멀티코어 환경에서는 여전히 다음 문제를 처리해야 한다.

- CPU별 부하 불균형
- 작업 Migration 비용
- 캐시 친화성
- CPU affinity
- NUMA 메모리 접근

따라서 EEVDF만으로 멀티코어 스케줄링의 모든 문제가 해결되는 것은 아니다.

[3] [CFS와 EEVDF 비교]

구분	기존 CFS	최신 EEVDF
목표	CPU 시간의 공정한 분배	공정한 분배와 응답성 개선
우선순위	nice 값과 가중치	nice 값과 가중치
기본 시간 정보	vruntime	vruntime
실행 자격 판단	별도의 명시적인 판단이 약함	lag를 이용해 판단
다음 작업 선택	가장 작은 vruntime	자격 있는 작업 중 가장 빠른 Virtual Deadline
짧은 작업 처리	공정하지만 응답성에 한계	짧고 지연에 민감한 작업에 유리
실시간 보장	없음	없음
복잡도	비교적 단순	상대적으로 복잡
주요 적용	과거 리눅스 일반 작업	Linux 6.6부터 전환된 최신 방식